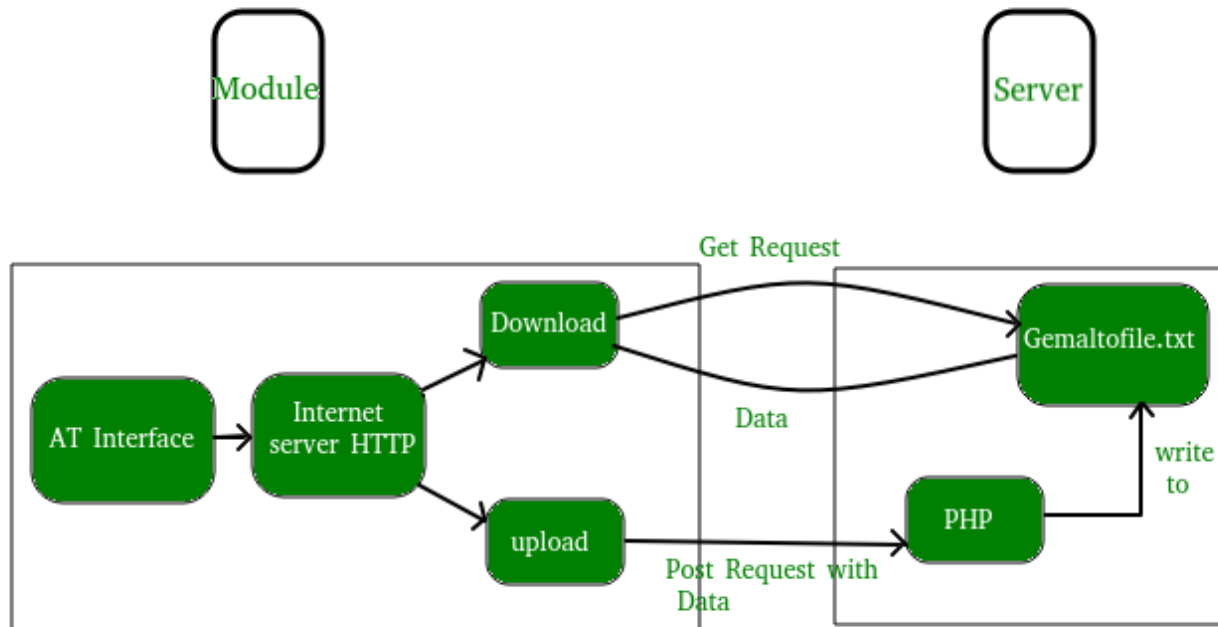

Welcome & Module Induction

- **What we learned previously:** In Module 9, we mastered Logging and Diagnostics, moving away from temporary prints to a professional system that tracks errors and manages disk space via rotating logs.
 - **What we are learning today:** We are connecting our Python scripts to the world. We will learn HTTP basics (GET, POST, PUT, DELETE), JSON manipulation, and how to securely access private data using API Keys and Tokens.
 - **Why this will be helpful:** In modern AI engineering, your code rarely works in isolation. You need to fetch live weather data, send images to a cloud-based GPU for processing, or interact with Large Language Models like OpenAI's GPT-4. Mastering the requests library allows you to build software that communicates across the entire internet.
-

HTTP Basics & The requests Library



- **The Concept:** HTTP is the language of the web. The requests library allows Python to "talk" to websites.
- **The Core Verbs:**
- **GET:** Asking the server to send you data (like reading a webpage).
- **POST:** Sending new data to a server (like submitting a login form).
- **PUT:** Updating existing data on a server.
- **DELETE:** Removing data from a server.

JSON: The Universal Data Language

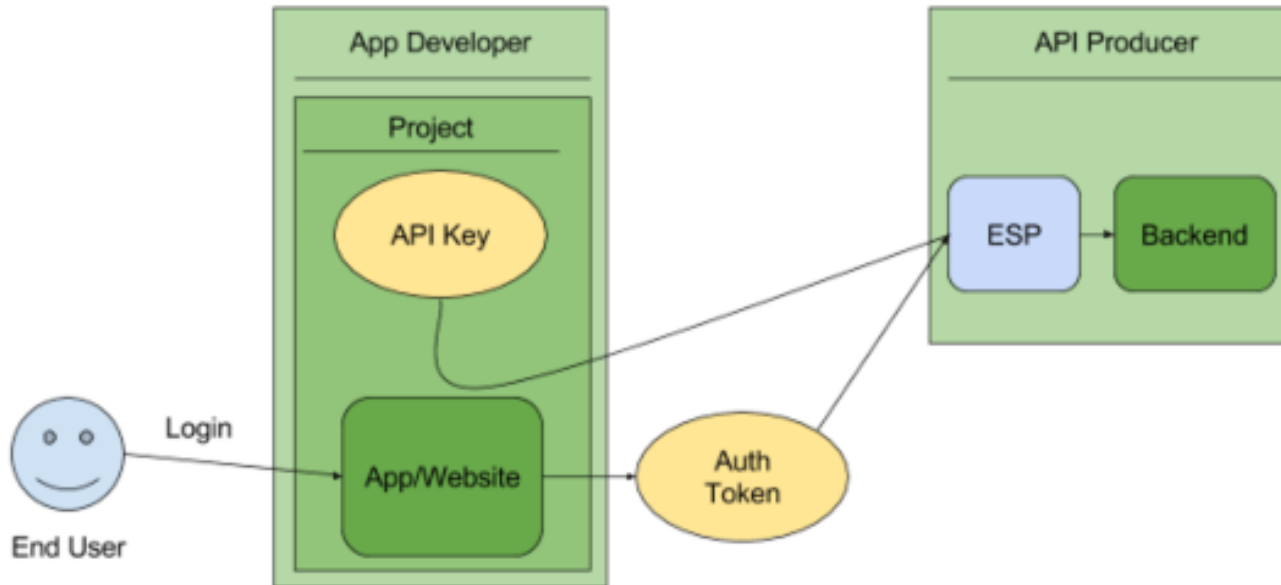
```
import json

student_data = {
    "name": "Ashton",
    "age": 15,
    "grade": 8,
    "subjects": ["math", "english", "science"]
}

f = open("output.json", 'w')
json.dump(student_data, f)
```

- **The Problem:** Python understands "Dictionaries," but a web server might be running Java or JavaScript.
- **The Solution:** JSON (JavaScript Object Notation). It is a simple text format that every programming language understands.
- **The Process:** We "serialize" (convert) Python dictionaries into JSON strings to send them away, and "deserialize" incoming JSON strings back into dictionaries to use them in our code.

API Authentication & Security



- **The Gatekeeper:** High-quality data (from Google or OpenAI) isn't free or public. You need permission to access it.
- **Authentication Methods:**
 - **API Keys:** A unique "password" for your script sent in the headers.
 - **Bearer Tokens:** A temporary, highly secure pass used for sensitive data.
- **Security Rule:** Never hardcode your API keys directly in the script; use environment variables to keep them hidden from GitHub.